

Introduction to Programming

Mengzhuo Guo

Academic Year 2026-2027

Contents

1 Course Information	1
2 Course Objectives	1
3 Introduction of Python	2
3.1 Attendance and Participation Policy	2
3.2 Assignment Policy	2
3.3 IMPORTANT: Academic Integrity and Use of AI Tools	2
4 Software and Learning Environment	2
5 Syllabus	2

1 Course Information

- **Course Name:** Introduction to Programming: Python
- **Instructor:** Mengzhuo Guo
- **Class Time:** Tuesday 19:00-22:00
- **Classroom:** To be announced
- **Contact:** mengzhguo@scu.edu.cn, or WeChat group
- **Course Website:** [Online Materials](#)

2 Course Objectives

Upon successful completion of this course, students should be able to:

- understand the fundamental syntax and concepts of Python;
- use conditional statements, loops, functions, and data structures to solve programming problems;
- apply exception handling, modules, and object-oriented programming in Python;
- independently design and develop simple Python programs;
- develop computational thinking and basic program debugging skills.

3 Introduction of Python

Python is a popular, high-level programming language. It is easy to learn because its syntax is simple, clear, and close to natural language.

Python supports object-oriented programming. Programmers can create classes and objects to organize code and make programs easier to reuse, maintain, and expand. It also supports other programming styles, such as procedural programming and functional programming.

Python is an interpreted and dynamically typed language, so programs can be written and tested quickly. It is also cross-platform, which means Python programs can run on Windows, macOS, and Linux. In addition, Python has a large number of libraries and frameworks for data analysis, artificial intelligence, web development, automation, and scientific computing.

Because of these features, Python is widely used by beginners, researchers, engineers, and software developers.

3.1 Attendance and Participation Policy

Attendance accounts for 10% of the final grade. Attendance may be checked randomly during class. Students who are absent, late, or leave early without permission may lose attendance points.

Students are encouraged to participate actively in class. Those who answer questions, raise thoughtful questions, contribute to discussions, or help explain programming ideas *may receive additional participation points*.

Students who are unable to attend class for a valid reason should inform the instructor in advance and provide relevant documentation when required.

3.2 Assignment Policy

There will be approximately five to six individual assignments during the semester. Assignments must be submitted before the announced deadline.

Late submissions may receive a grade penalty unless prior permission has been granted. Students may discuss general ideas with classmates, but all submitted code and written explanations must be completed independently.

Submitted programs should run correctly in the required Python environment. Students are responsible for checking their files before submission.

3.3 IMPORTANT: Academic Integrity and Use of AI Tools

Students must complete all individual assignments independently. Copying code from classmates, websites, or other sources without proper acknowledgment *is not permitted*.

AI tools may be used to support learning, such as explaining concepts, identifying errors, or providing suggestions. However, students must understand, verify, and be able to explain all submitted code. Directly submitting AI-generated solutions without understanding or modification may be treated as academic misconduct.

The instructor may ask students to explain their code when necessary.

4 Software and Learning Environment

Students are *expected* to install the following software before the second class:

- Python 3.10 or later;
- Anaconda;
- Jupyter Notebook;

All assignments should be completed using Python 3 unless otherwise specified.

5 Syllabus

Table 1: Course Schedule and Useful Learning Resources

Class	Topic	Main Contents	Useful URLs
Class 1	Introduction to Python	Course introduction; applications and characteristics of Python; installation of Python and Anaconda; Jupyter introduction	Anaconda Download Introduction to Python Jupyter Documentation
Class 2	Fundamentals of Python	Basic syntax; variables and constants; identifiers; comments; input and output; basic operators and expressions.	Python Basics Python Installation Environment Python Development Tools
Class 3	Program Flow Control I	Boolean expressions; comparison and logical operators; conditional statements; if , elif , and else ; related exercises.	Python Control Flow
Class 4	Program Flow Control II	for loops; while loops; break , continue , and pass ; nested loops and related exercises.	Python Control-Flow Tools Python Basic Tutorials
Class 5	Common Built-in Data Types I	Numbers; Boolean values; strings; string indexing, slicing, formatting, and common string methods; related exercises.	Python Built-in Types Python Strings
Class 6	Common Built-in Data Types II	Lists and tuples; indexing and slicing; common methods; Dictionaries and sets; creation, access, modification, deletion, traversal, and commonly used methods; mutable and immutable objects; Nested data structures; conversion between data types; selection of suitable data structures; related exercises.	Python Lists Python Data Structures Dictionaries and Sets Python Data Structures List Comprehensions
Class 7	Errors and Exception Handling	Syntax errors and runtime errors; try , except , else , and finally ; raising exceptions and designing exception-handling programs.	Errors and Exceptions
Class 8	Functions I	Function definition and invocation; return values; positional arguments; keyword arguments and default arguments; related exercises.	Defining Functions
Class 9	Functions II	Variable-length arguments; variable scope; local and global variables; recursive functions; documentation strings and related exercises.	Function Arguments and Scope
Class 10	Functional Programming and Exercises	Lambda expressions; functions as objects; map , filter , and reduce ; comprehensive exercises on functions.	Lambda Expressions Functional Programming

Continued on next page

Table 1 (continued)

Class	Topic	Main Contents	Useful URLs
Class 11	Object-Oriented Programming I	Basic concepts of object-oriented programming; classes and objects; attributes and methods; constructors and the <code>self</code> parameter; Encapsulation; inheritance; method overriding; polymorphism; class attributes and instance attributes; related exercises.	Python Classes Inheritance and Polymorphism
Class 12	Modular Programming	Modules and packages; importing modules; <code>import</code> and <code>from</code> ; the <code>__name__</code> variable; creating and using custom modules.	Python Modules and Packages